

# PART 6: EXECUTION & DELIVERY EXCELLENCE

## 6.0 Why this is the longest chapter in v2

---

Execution is where most PMs lose most often. They have great instincts, decent strategy, ambitious roadmaps — and they fail at the day-to-day of getting things shipped to high quality with the team intact at the end. The earlier chapters are required to be a PM. This chapter is required to be a PM who keeps the job.

Read it slowly. Do the exercises. The frameworks here are tested in real engineering rooms.

## 6.1 What execution actually is

---

Execution is the discipline of turning a decision into a shipped, working product, on time, at the quality bar, without burning out the team or breaking trust with stakeholders.

It is mostly invisible when done well. Done badly, it is what makes PMs hated.

Three principles govern execution:

1. **Clarity reduces friction.** Every decision the team has to make twice is friction. Every requirement misunderstood is rework. Clear PRDs, clear decisions, clear status produce more output than unclear versions even when the unclear versions seem faster.
2. **Trust is the bandwidth multiplier.** A team with high mutual trust ships more than the same team with low trust, with the same hours invested. Trust is built through reliability, transparency, and respect — over time.
3. **Outcomes over output.** Shipping features that don't move metrics is not execution. It's activity. Real execution is judged by what changed in the world after the team was done.

## 6.2 The PRD — a craft, not a template

---

The Product Requirements Document is the most-recognized PM artifact, and the most often misused.

A PRD is not:

- A specification of every implementation detail.
- A wishlist of features.
- A document written once and forgotten.

- A formality required by process.

A PRD *is*:

- A decision document. It captures decisions about what to build and why.
- A communication tool. It aligns engineering, design, and stakeholders.
- A reference. It is consulted during build, debate, and post-mortem.
- A trace. It records reasoning, so future teams can audit.

If your PRD doesn't serve those four functions, you are producing template fillings, not PRDs.

## Modern PRD structure (15 sections)

1. ONE-LINER
2. CONTEXT
3. PROBLEM STATEMENT
4. USERS AND USE CASES
5. PROPOSED SOLUTION
6. SUCCESS METRICS
7. NON-GOALS
8. EDGE CASES
9. RISKS AND MITIGATIONS
10. DEPENDENCIES
11. ROLLOUT PLAN
12. EXPERIMENTS / VALIDATION
13. OPEN QUESTIONS
14. APPENDICES
15. SIGN-OFFS

Each section prevents specific failure modes:

**One-liner:** Two sentences someone five layers above can understand. If you can't write it, you don't have clarity. Stop. Get clarity.

**Context:** Why this is on the table now. Prevents the most common stakeholder pushback: "why this and not something else?"

**Problem statement:** With evidence — interviews, data, support tickets, session recordings. A weak problem statement: "Users want better search." A strong one: "70% of customers rate search 'frustrating.' 'Can't find X' is 22% of support tickets. Users average 3.2 search attempts before finding what they want; 28% give up entirely."

**Users and use cases:** Specific personas with priority. Top 3-5 concrete scenarios. Discipline of writing concrete scenarios prevents the trap of designing for "users in general."

**Proposed solution:** What you're building, at the level engineers can implement. Engineers solve the implementation; you solve the intent.

**Success metrics:** Primary metric with target, secondary metrics as guardrails, negative metrics to watch, evaluation timeframe. Without metrics, the launch is unaccountable.

**Non-goals:** What this is NOT. What's out of scope. The most useful section because it prevents scope creep mid-build. Most PRDs skip it.

**Edge cases:** Empty states, error states, loading states, permission edges, network failure, concurrent access, data quality, high-load. Underspecified edge cases lead engineers to invent default behavior that's rarely right.

**Risks and mitigations:** Top 3-5. Likelihood × impact × mitigation. Senior PMs read this section first. They want to know: have you thought adversarially?

**Dependencies:** Other teams, systems, vendors with specific deliverables, deadlines, contacts. Dependencies are the #1 source of slip. Track them explicitly.

**Rollout plan:** Internal alpha → closed beta → percentage rollout → full launch. Specific feature-flag strategy.

**Experiments / validation:** A/B test design (hypothesis, variants, success criteria, sample size, decision rule), or explicit reasoning for why you're not testing.

**Open questions:** Items still being worked. Decisions deferred. Honesty here builds trust.

**Appendices:** Links to research findings, design files, technical docs, prior PRDs, customer quotes, dashboards.

**Sign-offs:** Names of reviewers and approvers. Stakeholders who must sign off.

## How long should a PRD be

As long as it needs to be. Small feature: 2-3 pages. Major launch: 10-20 pages. Strategic initiative: 30+ pages. The wrong frame is "shorter is better." The right frame is "the PRD must serve its four functions." If the PRD is 1 page and the team has 20 questions during implementation, the PRD failed. If it's 30 pages and only 4 are useful, it also failed.

(For a complete sample PRD walking through all 15 sections — the SmartSave Indian fintech example — see the v1 of this chapter; it's reproduced in the appendix of this book.)

## 6.3 Working with engineering — the hard cases

---

The v1 was too soft. Here's the harder version.

### The five recurring PM-engineering tensions

**Tension 1: PM thinks something is simple; engineering says it's hard.**

You ask for "just a checkbox to opt out of marketing emails." Engineering says 3 weeks. You feel they're padding.

What's actually happening: the checkbox you're imagining requires changes in 6 systems (user prefs DB, email service, transactional/marketing classification, audit log, account settings UI, mobile parity). Engineering isn't padding; you didn't see the iceberg.

Resolution: ask, don't push. "Walk me through the 3 weeks." If their breakdown makes sense, accept and learn. If parts don't make sense, push back specifically on those parts.

The PM who learns to recognize when "simple" is iceberg estimates engineering effort within 30% in 2 years. The PM who stays surface-level estimates within 200% forever.

### **Tension 2: Engineering wants to refactor; PM wants new features.**

The team wants a sprint on tech debt. You're under pressure to ship a customer-promised feature. You feel engineering is hiding behind tech work to avoid customer pressure.

What's actually happening (usually): the codebase is genuinely deteriorating. Every new feature is taking 1.5-2x longer than it should.

Resolution: respect the engineering judgment. Tech debt described as "we should refactor when we get a chance" is optional. Tech debt described as "we are losing time on every feature" is not optional. Trust the senior engineer's read. If you suspect the request is unjustified, ask for data: "What features last quarter were slowed by this debt?" If they answer specifically, fund it.

### **Tension 3: Engineering wants to over-engineer; PM wants to ship.**

You want a v1 that works. Engineering wants to build for scale, fault tolerance, and 5 future use cases.

Align on stage. "Is this a v1 we expect to throw away, or v1 we expect to scale?" If throw-away, push hard for simplicity. If foundational, the engineer's instincts are usually right.

Watch the trap: 80% of "throw-away v1" code lives forever. Calibrate the "throw-away" framing carefully.

### **Tension 4: PM disagrees with engineering's technical decision.**

Three questions to ask yourself:

1. **Do I have technical standing?** A non-technical PM weighing in on database choice is mostly noise. A technical PM weighing in on API surface area has weight.
2. **What's the user-facing impact?** If the architecture decision affects latency, cost, scaling, or reliability in user-visible ways, you have a stake. If it's purely engineering elegance with no user-visible consequence, defer.
3. **Have I escalated thoughtfully?** Going around the engineer to their manager is destructive. Going to the engineer with "walk me through the tradeoff" is constructive.

Most PM-engineering technical disagreements should resolve in engineering's favor unless the user-facing impact is concrete. Pick your battles.

### **Tension 5: Engineering reports the work is "almost done" — repeatedly.**

If "almost done" has been the answer for three weeks, you have a problem. Either: (a) the engineer underestimated and is afraid to say so, (b) edge cases are emerging unanticipated, (c) there's a hidden architectural problem, or (d) the engineer is blocked on something they haven't raised.

Ask specific questions: "What's the remaining work, exactly? What's been merged? What's open? What's blocked?" If you can't get specific answers, escalate to the EM (not over the engineer's head, but in their presence).

Your job is to make blockers visible. Engineers sometimes hide blockers because admitting them feels like failure. Make admission cost-free.

## Investing in the relationship

- **Read code occasionally.** Not to write it. To follow what's happening at high level.
- **Defend their estimates** when sales or executives push for unrealistic timelines.
- **Take the blame publicly** when something ships flawed. "I should have caught this in the PRD."
- **Celebrate engineers specifically** by name, not "the team."
- **Bring users to engineers.** Invite them to research calls.
- **Don't bring half-baked specs.** Spec quality is a sign of respect.

## 6.4 The PM-EM-Designer triangle and how it fights

---

PM owns what gets built and why. EM owns engineering capacity and quality. Designer owns the experience.

Each pulls in a different direction by default:

- PM toward outcomes (what moves the metric).
- EM toward sustainability (what the team can maintain).
- Designer toward craft (what's beautiful and usable).

### Common triangle conflicts

**Conflict 1: PM proposes feature X. Designer hates it. EM is neutral.**

The designer thinks the feature is bad UX. PM thinks it's necessary for revenue. They circle.

Resolution patterns:

- The designer's job is to make the feature as good as possible *if* it's built. Their job is not to veto strategic decisions.
- The PM's job is to listen and either incorporate concerns or explicitly accept the UX cost.
- "We'll build this with the worse UX, and revisit in 6 months when we have data" is a legitimate compromise.

The good PM acknowledges: "I hear you that this is bad UX. I think we need to ship it because of revenue pressure. Will you help me design the v1 in a way we can credibly improve later?"

**Conflict 2: EM says "engineering quality requires we don't ship until it's right." PM says "we promised customers."**

EM is protecting the team and the codebase. PM is protecting customer relationships. Both are right within their lens.

The EM's right is veto on quality, not on timing. They can say "this isn't ready"; they can't say "we'll ship when we feel like it." The PM's right is to push for velocity, not override quality judgments.

Escalation is sometimes correct. The Director of Engineering and Director of Product should make the call together. If escalation is required, that's a sign the situation is genuinely contested, not a sign anyone is being unreasonable.

The good PM says: "Walk me through specifically what's not ready. What's the actual customer-facing risk?" If the EM can specify, decide together. If they can't specify, gently push for shipping.

**Conflict 3: Designer wants to redesign; PM and EM want to keep building.**

The designer joins, sees the existing UI, and wants to redesign. PM wants to ship features. EM doesn't want to repaint a moving train.

Redesigns are strategically important but tactically risky. They take longer than estimated, often disrupt usage patterns, and rarely produce immediate metric wins. Right answer depends on stage:

- Pre-PMF: don't redesign; ship features.
- Post-PMF: occasional strategic redesigns.
- Mature product: redesigns can be necessary but require rigor.

Compromise: roadmap quarterly redesign budgets. Major redesign every 18-24 months. Component-level redesigns more frequently.

**Conflict 4: Disagreement on user research interpretation.**

PM and designer attended the same user interviews; draw different conclusions. Re-watch the interviews together. Identify specific moments where each person sees their conclusion. Often, both are partially right. The discipline is moving from competing interpretations to integrated synthesis.

**Conflict 5: PM is skipping design.**

PM ships features without bringing design in early. Designer feels reduced to "make it pretty after the fact." Quality suffers.

Structurally, design should be involved at problem definition, not at solution polish. The PM who skips this saves time short-term but pays in quality long-term.

## The healthy triangle

In a healthy triangle:

- PM is connective tissue, integrating perspectives.
- EM has veto on engineering quality and capacity.
- Designer has veto on design craft.
- Decisions are made by consensus on most things, with the PM breaking ties when necessary on direction.
- Disagreements are aired in the meeting, not after.
- Trust is high enough that "disagree and commit" works.

Building this takes 6-12 months. If you join a team with broken dynamics, spend 90 days observing. Identify the specific dysfunction (usually one or two patterns, not generalized). Then address the pattern, not the personalities.

## 6.5 Working with researchers — separately from designers

---

User researchers are a separate function from designers, with separate skills. As you progress, you'll work with dedicated researchers more often.

### What researchers actually do

A user researcher:

- Designs research studies (interview protocols, surveys, usability tests).
- Recruits participants.
- Conducts research professionally (without leading questions).
- Analyzes findings.
- Reports back with synthesis.

The skills are real. A great researcher will see things you don't, hear things you don't, ask questions you wouldn't have thought to ask.

### How PMs misuse researchers

- **Treating them as note-takers.** Researchers are not note-takers. If you treat them as such, they leave.
- **Briefing them poorly.** Vague questions produce vague research. Spend time on the brief.
- **Ignoring inconvenient findings.** When a researcher tells you something inconvenient, listen. They're often right.
- **Going around them.** Skipping the researcher to do your own interviews. Sometimes acceptable, often produces worse results.

## How to brief a researcher

Include:

- The decision you're trying to make.
- What you don't know that's blocking the decision.
- Who you want to learn about (specific segments).
- Hypotheses (so they can test rather than fish).
- Constraints (timeline, budget, available participants).
- Form of output you want (memo, presentation, themes, video clips).

The brief is a contract. With a good brief, the researcher produces what you need. Without one, they guess.

## How to use research findings

- Listen first. Don't argue findings.
- Ask clarifying questions about specifics ("when User 4 said X, what was the context?").
- Ask about confidence in conclusions.
- Don't conflate "users said X" with "we should do X." Research informs decisions; it doesn't make them.
- Synthesize together with the team. Different roles see different patterns.

## 6.6 Decision documents — the high-leverage PM artifact

---

Beyond PRDs, mature PMs produce decision documents — focused artifacts that capture a specific decision, its reasoning, and its outcome.

A decision document is shorter than a PRD (1-3 pages) and serves a different purpose. The PRD specifies what to build. The decision document records why we chose this and not that.

### When to write one

- Major scope or strategy decisions ("we will sunset this product line").
- Architectural decisions with long-lasting effects.
- Pricing decisions.
- Hiring or org structure decisions.
- Vendor or partner decisions.
- Anytime you'll want to remember why we decided this in 2 years.

## Structure

DECISION: [What was decided, in one sentence]  
DATE: [When]  
DECIDED BY: [Who held the pen; who signed off]

CONTEXT: [Why this decision was on the table]

OPTIONS CONSIDERED:

Option A: [...] – Pros, Cons, Cost  
Option B: [...] – Pros, Cons, Cost  
Option C: [...] – Pros, Cons, Cost

DECISION: [Which option, with rationale]

KEY ASSUMPTIONS: [What has to be true for this to work]

WHAT WOULD CHANGE THE DECISION: [What evidence would flip us]

RISKS AND MITIGATIONS: [Top 3]

EVALUATION DATE: [When we'll revisit]

EVALUATION CRITERIA: [What we'll look at]

## Worked example: pricing decision

DECISION: Increase Pro tier pricing from \$20/mo to \$30/mo, effective Q3.

DATE: April 14, 2026

DECIDED BY: PM (lead), Marketing Director, Head of Product (signed off)

### CONTEXT:

Pro tier pricing has been unchanged for 3 years. Competitor analysis shows median competitor at \$28-35/mo for equivalent feature set. Internal cost analysis shows our gross margin on Pro has dropped from 78% to 64% as AI costs increase. Customer surveys (N=240) show 18% rating "value for price" as poor; 71% rating fair-to-excellent.

### OPTIONS CONSIDERED:

- A: Hold price at \$20. Pros: no churn risk, no comms burden. Cons: margin continues to compress; competitive positioning suffers.
- B: Increase to \$25. Pros: modest lift, manageable comms. Cons: still below competitor median; margin recovery insufficient.
- C: Increase to \$30. Pros: aligns with competitor pricing, restores margin to 73%. Cons: 30% increase carries churn risk; comms burden significant.
- D: Increase to \$35. Pros: position as premium. Cons: above competitor median; high churn risk; doesn't match feature parity.

DECISION: Option C (\$30/mo).

### RATIONALE:

- \$30 aligns with competitor median, ensuring we're not perceived as overpriced.
- Restores margin to 73%, sustainable for projected AI cost trajectory.
- Modest 50% increase requires careful comms but is defensible given 3-year price hold.
- Existing customers grandfathered for 12 months at current price; reduces immediate churn risk.

### KEY ASSUMPTIONS:

- Churn from new customers acquiring at \$30 is <15% above baseline.
- AI cost trajectory continues to require ongoing margin support.

### WHAT WOULD CHANGE THE DECISION:

- If new-customer churn at \$30 exceeds 25% (vs ~10% baseline), rollback to \$25.
- If competitors pricing-cut materially below \$25, reassess.

### RISKS AND MITIGATIONS:

1. Churn higher than expected. Mitigation: rollback plan ready, monitor weekly, decision point at week 6.
2. Negative press / community reaction. Mitigation: comms plan with transparent rationale; CEO involved in messaging.
3. Sales cycles lengthen. Mitigation: ROI calculator updated; sales training before launch.

EVALUATION DATE: October 14, 2026 (6 months post-launch).

EVALUATION CRITERIA: New-customer acquisition rate, churn rate, total revenue, NPS, community sentiment.

## Why decision documents matter

1. **Audit trail.** When something goes wrong (or right) in 2 years, you can reconstruct the reasoning.

2. **Avoid re-litigation.** New team members ask "why did we decide this?" and you point to the doc.
3. **Force clarity at decision time.** Writing surfaces ambiguities you'd miss orally.
4. **Build organizational memory.** Decisions become artifacts the company learns from.
5. **PM signal.** PMs who produce decision documents look more senior than PMs who don't.

## 6.7 The Big Tech launch process — Google specifically

---

In v1, I described general launch playbooks. Here's the more specific Big Tech version.

### Google's launch process

For any user-facing launch at Google, the process goes through (roughly):

#### 1. Product readiness. PM and team confirm:

- Functional completeness vs PRD.
- Performance benchmarks met.
- Accessibility audit passed.
- Internationalization tested.
- Edge cases handled.

#### 2. Launch readiness review (LRR). Multi-functional review covering:

- Product readiness.
- Technical readiness (load testing, monitoring, on-call).
- Privacy review (legal sign-off on data handling).
- Security review.
- Legal review (terms of service, jurisdiction).
- Marketing readiness.
- Communications readiness (PR, blog, customer comms).

#### 3. Launch calendar slotting. Major launches are slotted on a company calendar to avoid conflicts (with other launches, with major events, with quiet periods).

#### 4. Approval chain. For significant launches, approvals required at multiple levels — director, VP, sometimes SVP. Each level has questions; PM answers them.

#### 5. Launch day execution. Coordinated rollout. Live monitoring. Cross-functional war room for major launches.

#### 6. Post-launch review. Within 30-60 days, retrospective. What hit, what missed, what we learned, what we'd change next time.

## What this means for the PM

The PM is the launch quarterback. You don't do every check yourself, but you ensure every check happens. Specifically:

- You schedule and run the launch readiness review.
- You answer privacy, security, legal questions (with help from those teams).
- You shepherd the approval chain.
- You coordinate launch day with eng, marketing, support.
- You run the post-launch review.

A botched launch readiness review at Google means the launch is delayed, sometimes by weeks. The PM is the one held accountable for the delay.

## The pace difference

A common observation from people moving Big Tech ↔ startup: launches at Big Tech take 6-12 weeks of process for what a startup would ship in 2 days. Both are correct for their context.

Big Tech process exists because:

- Reach is enormous (a Google bug affects billions).
- Brand reputation is fragile.
- Regulatory scrutiny is heavy.
- Coordination across teams is required.

Startup speed is correct because:

- Reach is small (you can recover from mistakes).
- The cost of slow learning is higher than the cost of small mistakes.

Don't import Big Tech process into a startup; it kills velocity. Don't import startup speed into Big Tech; it gets you fired.

The PM who has worked in both can choose the right cadence. That's a 5-10 year skill.

## 6.8 Working with sales, marketing, support, legal, finance

---

Brief notes; v1 covered most of these.

**Sales:** Treat them as customer signal, not as feature dictators. Sales-driven roadmaps fail because they over-rotate on the loudest customer. PM filters: this complaint, is it from many customers or one? Is it about a real product gap or a one-off?

**Marketing:** Bring them in early. Marketing involved at PRD stage produces better launch messaging than marketing involved at launch readiness. They also surface positioning issues you'd miss.

**Customer support:** Underrated PM input source. Support sees every bug, every feature gap, every confusion. Build a weekly digest from support → product. The pattern of complaints is gold.

**Legal:** For consumer products, involve legal at PRD stage when handling user data, payments, content moderation, or anything regulatory. For B2B, involve them at contract negotiation. Late legal review delays launches more than any other dependency.

**Finance:** For pricing decisions, monetization changes, or anything affecting revenue model. Finance produces the unit economics; PM produces the strategic recommendation; together you decide.

## 6.9 Agile, Scrum, Kanban — what matters, what doesn't

---

Brief refresh from v1, with sharper opinions.

The Agile Manifesto (2001) listed values:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

These are correct. The implementations that grew up around them (Scrum, SAFe, etc.) are mostly process theater that violates the spirit of the manifesto.

### Scrum reality

Scrum's prescriptions: sprints (2-week iterations), daily standups, sprint planning, sprint review, sprint retrospective. Roles: Product Owner, Scrum Master, Development Team.

What's right about Scrum: the cadence of regular planning + execution + review + retrospective. This rhythm produces learning.

What's wrong: the formality. Story points obsessions. Sprint commitment ceremonies. Velocity measurement. Backlog grooming as a meeting. Daily standups that go 30 minutes.

Most strong product teams use Scrum loosely — they have a rhythm of iteration and review, but they don't do all the ceremony. The ceremony is for teams that need scaffolding because they don't have intrinsic discipline.

## Kanban

Continuous flow rather than time-boxed sprints. Work moves through columns (backlog → in progress → review → done) at the pace of completion. WIP limits prevent overload.

Kanban works well for teams whose work is irregular (bug fixes, ops, support). Less well for teams whose work has clear time-bounded scopes.

## Hybrids

Most modern teams hybrid: Kanban-style continuous flow with weekly or bi-weekly demos and retros. Less ceremony than Scrum, more rhythm than pure Kanban.

## What actually matters

Not the ceremony, but:

- The team is shipping regularly.
- The team is learning from what they ship.
- Decisions can be made fast.
- People know what they're doing this week and why.
- Blockers surface fast.
- Quality is monitored.

If those things are happening, your process is fine, regardless of what you call it. If they're not happening, no process will save you.

## 6.10 Anti-patterns

---

A list of execution failures to avoid:

1. **Story-point obsession.** Teams arguing about whether something is 5 or 8 points. Story points are coordinates for relative effort, not a metric. Don't optimize them.
2. **Backlog hoarding.** Backlogs of 1,000+ items. Most are noise. Cull aggressively.
3. **Specification by ticket.** Trying to capture all requirements in JIRA tickets. JIRA is for tracking, not specifying. Use real PRDs.
4. **Commitment ceremony theater.** Sprint planning where the team "commits" to scope they don't believe in. Resentment builds. Trust erodes.
5. **Velocity as a metric.** Measuring teams by velocity creates incentive to inflate estimates. Measure outcomes, not velocity.
6. **All-hands sprint reviews.** 50 people watching a demo. Most don't care. Cut.
7. **Retros that change nothing.** Same issues raised every retro. Nothing changes. Retros without follow-through corrode team morale.

8. **Status meetings instead of status documents.** Information that could be a 2-minute read becomes a 30-minute meeting. Default to written; meet only when needed.

## 6.11 Common delivery risks and mitigations

---

Six risks every PM should anticipate:

1. **Scope creep.** Mitigation: explicit non-goals. Re-confirm scope when changes are proposed. Trade-off conversations: "to add X, we'd defer Y."
2. **Dependency slip.** Mitigation: track dependencies explicitly with deadlines and contacts. Weekly check-ins on dependency status.
3. **Engineering estimate breach.** Mitigation: track estimate accuracy over time per engineer; calibrate. Add buffer. Surface slips early.
4. **Quality regression.** Mitigation: quality gates before merge. Automated regression tests. Manual QA for critical paths.
5. **Last-minute legal/privacy block.** Mitigation: involve legal/privacy at PRD stage, not at launch readiness.
6. **Stakeholder surprise.** Mitigation: weekly updates. Pre-wire major decisions. No major news in unscheduled meetings.

## 6.12 Pre-launch checklist

---

A comprehensive checklist for launching:

### Functional:

- ☐ PRD fully implemented (not just "mostly").
- ☐ All edge cases handled or explicitly accepted.
- ☐ All known bugs triaged (P0/P1 fixed; P2/P3 documented).
- ☐ Cross-platform parity confirmed (iOS, Android, web).
- ☐ Localization tested for target markets.
- ☐ Accessibility audit passed.

### Technical:

- ☐ Load testing passed at expected scale.
- ☐ Monitoring and alerting configured.

- ☐  
On-call schedule confirmed.
- ☐  
Rollback plan documented and tested.
- ☐  
Feature flag configured.
- ☐  
Performance benchmarks met.

**Legal/privacy/security:**

- ☐  
Privacy review signed off.
- ☐  
Security review signed off.
- ☐  
Legal review signed off (terms, jurisdictions).
- ☐  
Data retention policies confirmed.
- ☐  
Audit logging configured.

**Comms/marketing:**

- ☐  
Launch blog post drafted.
- ☐  
Press release (if applicable) drafted.
- ☐  
Social posts scheduled.
- ☐  
Email to existing users prepared.
- ☐  
Help docs updated.
- ☐  
In-app announcements ready.

**Sales/support readiness:**

- ☐  
Sales team trained on new feature.
- ☐  
Battle cards updated.
- ☐  
Support team trained on FAQs.
- ☐  
Pricing pages updated.
- ☐  
Onboarding flows updated.

**Metrics/analytics:**

- ☐ Instrumentation in place.
- ☐ Dashboards built.
- ☐ Success criteria defined.
- ☐ Decision rule for rollback documented.

If you have all of these checked before launch, you've done the job. Most launches that go badly are missing 3-5 of these.

## 6.13 Practice exercises for Part 6

---

### Exercise 6.1 — Write a complete PRD

For a hypothetical feature you'd add to a product you use, write a complete PRD using all 15 sections. Aim for 8-12 pages. Have a friend review.

### Exercise 6.2 — Decision document

Pick a real decision in your career (kept or proposed). Write a decision document for it. If you can't fill in all sections, that's information about decision quality.

### Exercise 6.3 — PM-EM-Designer triangle audit

For a current or recent project, identify which of the 5 triangle conflicts you experienced. How was it resolved? What would you do differently next time?

### Exercise 6.4 — Engineering estimate calibration

For 5 features your team has shipped, compare estimated vs actual time. Compute the average overrun. Is it 30%? 100%? 200%? Use this to calibrate future estimates.

### Exercise 6.5 — Bad PRD diagnosis

Find a real-world PRD (your team's, a public one). Score it against the 15 sections. Rewrite the weakest section.

### Exercise 6.6 — Launch readiness simulation

For a hypothetical launch, work through the entire pre-launch checklist (6.12). Identify which items are unfamiliar to you. Research and fill the gaps.

## Exercise 6.7 — Researcher brief

Write a research brief for a research question you actually have about your product. Include all elements from 6.5. Pretend you're handing it to a senior researcher.

## Exercise 6.8 — Anti-pattern recognition

For your current team, score 1-5 on each of the 8 anti-patterns in 6.10. Identify the worst one. Propose one specific change to address it.

## 6.14 If you remember only one thing from Part 6

**Execution is mostly invisible when done well; loud when done badly. The discipline that produces invisible-when-done-well is: clarity at the start (PRDs), rigor in the middle (tracking dependencies, surfacing risks), honesty at the end (post-launch retros). PMs who skip any of three are the ones who become loud-when-done-badly. The PMs who internalize all three become indispensable.**

## 6.15 Google Lens — what execution interviews test

Execution questions in Big Tech interviews ("you're behind on a launch — what do you do?", "engineering says it's not ready — how do you handle?", "stakeholders disagree — walk me through your approach") test:

1. **Structured response under pressure.** Can you decompose the situation?
2. **Calibration.** Do you treat ambiguous information ambiguously, or pretend certainty?
3. **Stakeholder awareness.** Do you understand what each party cares about?
4. **Trade-off reasoning.** Do you make the costs explicit?
5. **Action orientation.** Do you commit to specific next steps?

Strong candidates name the trade-offs explicitly and commit to specific actions. Weak candidates list considerations without committing.

In execution interviews specifically, the interviewer is often watching for: would I want this person on my team? The behavioral signal of "this person is reliable, communicates well, doesn't dodge" matters more than the analytical answer.

---

*End of Part 6, v2.*

\newpage